We apply multiple prompting strategies to generate natural-language descriptions for each method node extracted from the Abstract Syntax Tree (AST). Specifically, we use a consistent prompt structure across zero-shot, few-shot (one-shot and two-shot), and chain-of-thought prompting to elicit responses from large language models (LLMs) to user queries in the dataset.

During inference, we retrieve the top-k candidate code chunks by measuring lexical similarity between the generated natural-language descriptions and the user query. These retrieved candidates are then provided to the LLM, which is prompted to identify the final corresponding code method. The prompt used for this final code-selection step is described separately.

Code Paper Pair Alignment Prompt:

SYS_PROMPT=You are an expert research/code auditor. Judge how strongly the Python file implements the specific methodology.DO NOT EXPLAIN ANYTHING. MUST Return ONE integer 1..5, or JSON {\"score\": <int>}."

Task: Output ONE integer 1..5 indicating how strongly the Python file implements the methodology.\n"

"Scale: 1=unrelated, 2=weak, 3=mixed, 4=good, 5=strong.\n"

"DO NOT EXPLAIN ANYTHING.MUST Return JUST the number, or JSON like {\"score\": <int>}."

(A) Methodology (paper):

{paper}

(B) Python file:

{code}

Best Candidate Selection(LLM as a Judge):

```
You are a code repository resolver.
User query:
{user_query}
Pick the single best matching candidate from the list.
If none match, set chosen_index=null and answer=null.
Return STRICT JSON matching this schema (types shown):
{json.dumps(schema, indent=2)}
Guidance:
- Use the candidate natural_description + signature + file_path.
- Do NOT include any extra keys outside the schema.
Candidates (JSON array):
{json.dumps(candidates, ensure_ascii=False)}
```

COT prompting:

"You generate a concise, search-friendly description of a Python function."

"Return ONLY one paragraph (2–5 sentences) describing:"

"- what the function does"

"- key inputs/outputs"

"- side effects (filesystem/network/db) if any"

"- notable edge cases or assumptions"

Think step-by-step privately to understand the function, but DO NOT output your reasoning.

Output ONLY the final one-paragraph description.

Signature:{signature}

Docstring:{docstring}

Code:{code}

Two Shot Prompting:

"You generate a concise, search-friendly description of a Python function."

"Return ONLY one paragraph (2–5 sentences) describing:"

"- what the function does"

"- key inputs/outputs"

"- side effects (filesystem/network/db) if any"

"- notable edge cases or assumptions"

Signature: {example_signature1}

Docstring: {example_docstring1}

Code: {example_code1}

Description: {example_description1}

Signature: {example_signature2}

Docstring: {example_docstring2}

Code: {example_code2}

Description: {example_description2}

Now do the same for:

Signature:{signature}

Docstring:{docstring}

Code:{code}

One shot Prompting:

"You generate a concise, search-friendly description of a Python function."

"Return ONLY one paragraph (2–5 sentences) describing:"

"- what the function does"

"- key inputs/outputs"

"- side effects (filesystem/network/db) if any"

"- notable edge cases or assumptions"

Think step-by-step privately to understand the function, but DO NOT output your reasoning.

Output ONLY the final one-paragraph description.

Signature:{signature}

Docstring:{docstring}

Code:{code}

Zero Shot Prompting:

"You generate a concise, search-friendly description of a Python function."

"Return ONLY one paragraph (2–5 sentences) describing:"

"- what the function does"

"- key inputs/outputs"

"- side effects (filesystem/network/db) if any"

"- notable edge cases or assumptions"

Signature: {signature}

Docstring: {docstring}

Code: {code}